Name : Mariam Mohamed Ramadan

Group :SE2

Lab 4

Part A :Project Folder Structure

```
mariamramadan@Ukhhh: ~/week4-lab
mariamramadan@Ukhhh:~$ mkdir -p week4-lab/product-service week4-lab/order-service
mariamramadan@Ukhhh:~$ cd week4-lab
mariamramadan@Ukhhh:~/week4-lab$ docker compose up --build -d
no configuration file provided: not found
mariamramadan@Ukhhh:~/week4-lab$ touch docker-compose.yml
mariamramadan@Ukhhh:~/week4-lab$ touch product-service/app.py product-service/requirements.txt product-service/Dockerfile
mariamramadan@Ukhhh:~/week4-lab$ touch order-service/app.py order-service/requirements.txt order-service/Dockerfile
mariamramadan@Ukhhh:~/week4-lab$ find .
.
./order-service
./order-service/app.py
./order-service/Dockerfile
./order-service/requirements.txt
./docker-compose.yml
./product-service
./product-service/app.py
./product-service/Dockerfile
./product-service/requirements.txt
mariamramadan@Ukhhh:~/week4-lab$
```

Part B: Product Service Implementation

mariamramadan@Ukhhh: ~/week4-lab

```
>     3: {"id": 3, "name": "Headphones", "price": 120}
> }
>
> @app.route("/health", methods=["GET"])
> def health():
>     return jsonify({"service": "product-service", "status": "up"}), 200
>
> @app.route("/products/<int:product_id>", methods=["GET"])
> def get_product(product_id):
>     product = PRODUCTS.get(product_id)
   if pr>     if product:
>         return jsonify(product), 200
>     return jsonify({"error": "Product not found"}), 404
>
> if __name__ == "__main__":
    app.>     app.run(host="0.0.0.0", port=5001)
> EOF
mariamramadan@Ukhhh:~/week4-lab$ cat product-service/app.py
from flask import Flask, jsonify

app = Flask(__name__)

PRODUCTS = {
    1: {"id": 1, "name": "Laptop", "price": 1200},
    2: {"id": 2, "name": "Phone", "price": 650},
    3: {"id": 3, "name": "Headphones", "price": 120}
}

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"service": "product-service", "status": "up"}), 200

@app.route("/products/<int:product_id>", methods=["GET"])
def get_product(product_id):
    product = PRODUCTS.get(product_id)
    if product:
        return jsonify(product), 200
    return jsonify({"error": "Product not found"}), 404

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5001)
mariamramadan@Ukhhh:~/week4-lab$
```

Activa

Part C: Order Service Implementation

```
mariamramadan@Ukhhh: ~/week4-lab
PRODUCT_SERVICE_URL = os.getenv("PRODUCT_SERVICE_URL", "http://product-service:5001")

def fetch_product(product_id, retries=2, delay=1):
    url = f"{PRODUCT_SERVICE_URL}/products/{product_id}"
    for attempt in range(retries + 1):
        try:
            response = requests.get(url, timeout=2)
            return response
        except requests.exceptions.RequestException:
            if attempt < retries:
                time.sleep(delay)
            else:
                return None

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"service": "order-service", "status": "up"}), 200

@app.route("/orders", methods=["POST"])
def create_order():
    data = request.get_json()
    product_id = data.get("product_id")
    quantity = data.get("quantity", 1)

    response = fetch_product(product_id)

    if response is None:
        return jsonify({"error": "product-service unavailable"}), 503

    if response.status_code != 200:
        return jsonify({"error": "invalid product"}), 400

    product = response.json()
    total = product["price"] * quantity

    return jsonify({
        "message": "Order created",
        "product": product["name"],
        "quantity": quantity,
        "total_price": total
    }), 201

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5002)
mariamramadan@Ukhhh:~/week4-lab$
```

Activate Windows
Go to Settings to activate W

Part D: Service Containerization

```
mariamramadan@Ukhhh: ~/week4-lab
    return jsonify({
        "message": "Order created",
        "product": product["name"],
        "quantity": quantity,
        "total_price": total
    }), 201

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5002)
mariamramadan@Ukhhh:~/week4-lab$ cat <<EOF > product-service/Dockerfile
ROM p> FROM python:3.12-slim
> WORKDIR /app
PY re> COPY requirements.txt .
> RUN pip install --no-cache-dir -r requirements.txt
OPY > COPY app.py .
> EXPOSE 5001
 ["pyt> CMD ["python", "app.py"]
> EOF
mariamramadan@Ukhhh:~/week4-lab$ cat <<EOF > order-service/Dockerfile
> FROM python:3.12-slim
> WORKDIR /app
> COPY requirements.txt .
> RUN pip install --no-cache-dir -r requirements.txt
> COPY app.py .
> EXPOSE 5002
> CMD ["python", "app.py"]
> EOF
mariamramadan@Ukhhh:~/week4-lab$ cat product-service/Dockerfile
cat order-service/DockerfileFROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5001
CMD ["python", "app.py"]
mariamramadan@Ukhhh:~/week4-lab$ cat order-service/Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5002
CMD ["python", "app.py"]
mariamramadan@Ukhhh:~/week4-lab$
```

Activat
Go to Se

Part E: Docker Compose Configuration

```
mariamramadan@Ukhhh: ~/week4-lab
>     container_name: order-service
>     ports:
>       - "5002:5002"
>     environment:
>       PRODUCT_SERVICE_URL: http://product-service:5001
>     depends_on:
>       - product-service
>     healthcheck:
>       test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:5002/health')"]
    int>       interval: 10s
>       timeout: 5s
>       retries: 3
>     restart: unless-stopped
> EOF
mariamramadan@Ukhhh:~/week4-lab$ cat docker-compose.yml
services:
  product-service:
    build: ./product-service
    container_name: product-service
    ports:
      - "5001:5001"
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:5001/health')"]
      interval: 10s
      timeout: 5s
      retries: 3
    restart: unless-stopped

  order-service:
    build: ./order-service
    container_name: order-service
    ports:
      - "5002:5002"
    environment:
      PRODUCT_SERVICE_URL: http://product-service:5001
    depends_on:
      - product-service
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:5002/health')"]
      interval: 10s
      timeout: 5s
      retries: 3
    restart: unless-stopped
mariamramadan@Ukhhh:~/week4-lab$
```

Part F: Build and run the system

```
 => => exporting config sha256:d4d64e8e89d2e82912ca817aa0f910e7f58732e311a32435ab7b843798258 6a3                    0.1s
 => => exporting attestation manifest sha256:e77541c4788072ad86d1fe4f51f139935c44c888c16a80f2c5e0f91915ae1a42       0.1s
 => => exporting manifest list sha256:af94ab1bc7f7966ce38c46603b72a9f49309438dd3cdfc806e0bf13088e0b957              0.1s
 => => naming to docker.io/library/week4-lab-order-service:latest                                                   0.0s
 => => unpacking to docker.io/library/week4-lab-order-service:latest                                                2.4s
 => [product-service] resolving provenance for metadata file                                                        0.2s
 => [order-service] resolving provenance for metadata file                                                          0.1s
[+] up 5/5
 ✔Image week4-lab-product-service Built                                                                            94.7s ✔Image week4-lab-order-service   Built
                                                                                                                   94.7s ✔Network week4-lab_default       Created
                                                                                                                          0.3s ✔Container product-service       Started
                                                                                                                                                                4.2s ✔Container order-servi
ce          Started                                                                                                                                   4.5smariamramadan@Ukhhh:~/week4-lab$ docker compose ps
NAME                IMAGE                       COMMAND           SERVICE           CREATED          STATUS                    PORTS
order-service       week4-lab-order-service     "python app.py"   order-service     42 seconds ago   Up 37 seconds (healthy)   0.0.0.0:5002->5002/tcp, [::]:5002->5002/tcp

product-service     week4-lab-product-service   "python app.py"   product-service   43 seconds ago   Up 38 seconds (healthy)   0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp

mariamramadan@Ukhhh:~/week4-lab$ docker compose logs product-service
er compose logs order-serviceproduct-service  |  * Serving Flask app 'app'
product-service  |  * Debug mode: off
product-service  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
product-service  |  * Running on all addresses (0.0.0.0)
product-service  |  * Running on http://127.0.0.1:5001
product-service  |  * Running on http://172.18.0.2:5001
product-service  | Press CTRL+C to quit
product-service  | 127.0.0.1 - - [22/Apr/2026 20:21:25] "GET /health HTTP/1.1" 200 -
product-service  | 127.0.0.1 - - [22/Apr/2026 20:21:36] "GET /health HTTP/1.1" 200 -
product-service  | 127.0.0.1 - - [22/Apr/2026 20:21:48] "GET /health HTTP/1.1" 200 -
product-service  | 127.0.0.1 - - [22/Apr/2026 20:21:59] "GET /health HTTP/1.1" 200 -
mariamramadan@Ukhhh:~/week4-lab$ docker compose logs order-service
order-service  |  * Serving Flask app 'app'
order-service  |  * Debug mode: off
order-service  | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
order-service  |  * Running on all addresses (0.0.0.0)
order-service  |  * Running on http://127.0.0.1:5002
order-service  |  * Running on http://172.18.0.3:5002
order-service  | Press CTRL+C to quit
order-service  | 127.0.0.1 - - [22/Apr/2026 20:21:27] "GET /health HTTP/1.1" 200 -
order-service  | 127.0.0.1 - - [22/Apr/2026 20:21:38] "GET /health HTTP/1.1" 200 -
order-service  | 127.0.0.1 - - [22/Apr/2026 20:21:50] "GET /health HTTP/1.1" 200 -
order-service  | 127.0.0.1 - - [22/Apr/2026 20:22:01] "GET /health HTTP/1.1" 200 -
order-service  | 127.0.0.1 - - [22/Apr/2026 20:22:11] "GET /health HTTP/1.1" 200 -
mariamramadan@Ukhhh:~/week4-lab$
```

Activate Windows
Go to Settings to activate Windows.

Part G: System Functional Testing

```
order-service   | 127.0.0.1 - - [22/Apr/2026 20:22:11] "GET /health HTTP/1.1" 200 -
mariamramadan@Ukhhh:~/week4-lab$ curl http://localhost:5001/health
{"service":"product-service","status":"up"}
mariamramadan@Ukhhh:~/week4-lab$ curl http://localhost:5001/products/1
{"id":1,"name":"Laptop","price":1200}
mariamramadan@Ukhhh:~/week4-lab$ curl http://localhost:5001/products/99
{"error":"Product not found"}
mariamramadan@Ukhhh:~/week4-lab$ curl -X POST http://localhost:5002/orders \
>  -H "Content-Type: application/json" \
>  -d '{"product_id": 1, "quantity": 2}'
{"message":"Order created","product":"Laptop","quantity":2,"total_price":2400}
mariamramadan@Ukhhh:~/week4-lab$ A
```

Part H: Failure Injection & Resilience Observation

```
mariamramadan@Ukhhh:~/week4-lab$ docker stop product-service
product-service
mariamramadan@Ukhhh:~/week4-lab$ curl -X POST http://localhost:5002/orders \
<!doctype html>
<html lang=en>
<title>415 Unsupported Media Type</title>
<h1>Unsupported Media Type</h1>
<p>Did not attempt to load JSON data because the request Content-Type was not &#39;application/json&#39;.</p>
curl: (3) URL rejected: Malformed input to a URL function
mariamramadan@Ukhhh:~/week4-lab$   -H "Content-Type: application/json" \
>   -d '{"product_id": 2, "quantity": 1}'
-H: command not found
mariamramadan@Ukhhh:~/week4-lab$ docker start product-service
product-service
mariamramadan@Ukhhh:~/week4-lab$ docker compose ps
NAME               IMAGE                      COMMAND            SERVICE            CREATED         STATUS                          PORTS
order-service      week4-lab-order-service    "python app.py"    order-service      7 minutes ago   Up 7 minutes (healthy)          0.0.0.0:5002->5002/tcp, [::]:5002->5
002/tcp
product-service    week4-lab-product-service  "python app.py"    product-service    7 minutes ago   Up 7 seconds (health: starting) 0.0.0.0:5001->5001/tcp, [::]:5001->5
001/tcp
mariamramadan@Ukhhh:~/week4-lab$
```

Part I :Architecture reflection task

1) Benefits of Microservices over Monolith The lab demonstrates separation of concerns; each service has a dedicated codebase and Dockerfile, allowing them to be developed, deployed, and scaled independently.

2) New Complexities Introduced Splitting the system adds overhead in managing network communication, service discovery, environment variables, and cross-service failure handling that wouldn't exist in a single monolith.

3) Impact of Latency and Slowness Because the services communicate synchronously, any latency or downtime in the `product-service` directly delays or breaks the `order-service` workflow, causing request timeouts or failures.

4) Visible 12-Factor App Principles The implementation follows 12-factor principles by using environment variables for configuration, maintaining a stateless design, and providing health endpoints for automated monitoring.